

Mid-Term Lab Examination — 8-bit Accumulator

University of Asia Pacific

Department of Computer Science and Engineering

Course Code : CSE 450 (VLSI Lab)

Mid-Term Lab Examination | Group C1 | Spring 2026

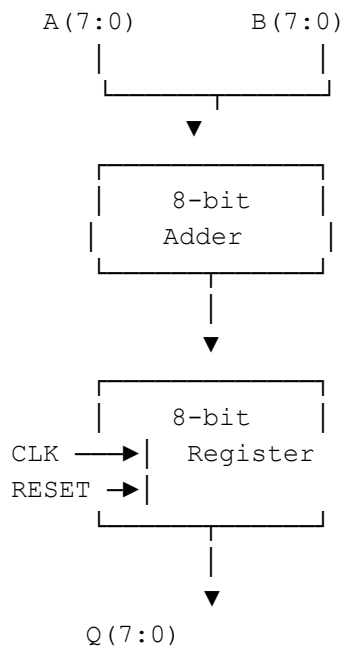
Duration: 1 hr Total Marks: 20

Student Name: Naima Tabassum Antora ID: 22201068

Mid-Term Task: 8-bit Accumulator

Objective. Design and implement an 8-bit accumulator in VHDL by combining the 8-bit adder and register concepts developed in the laboratory. The system must perform addition synchronously with the clock and store the result in an 8-bit register.

System Architecture



Functional requirement.

At every rising edge of CLK, the register stores the sum of the two 8-bit inputs. RESET has priority over the addition operation.

Required Operation

RESET	CLK	A, B	Operation / Q
1	↑	XX, XX	$Q \leftarrow 00000000$
0	↑	A, B	$Q \leftarrow A + B$
0	No rising edge	XX, XX	Q remains unchanged

Required Test Sequence

Your testbench must produce the following sequence. The expected Q value should be observed after the corresponding rising clock edge.

Step	RESET	A	B	Operation	Expected Q / Explanation
1	1	00000000	00000000	Reset	00000000 — Register is cleared
2	0	00000011	00000101	3 + 5	00001000 — Load sum
3	0	00000010	00000001	2 + 1	00000011 — Load new sum
4	0	11111111	00000001	255 + 1	00000000* — 8-bit overflow
5	0	10101010	00000101	170 + 5	10101111 — Load new sum

* Note: For Step 4, the mathematical result is 256, but an 8-bit output can represent only 0–255. Therefore, Q = 00000000 and the carry-out is discarded unless your implementation explicitly exposes it.

Implementation Requirements

- Use VHDL and IEEE.STD_LOGIC_1164.ALL.
- The register operation must be synchronized with the rising edge of CLK.
- RESET must be synchronous and active-high.
- Do not use a behavioral integer-based accumulator as a replacement for the required adder/register structure.
- The design should be modular and clearly identify the 8-bit adder and 8-bit register portions.
- The submitted testbench must generate a clock and apply the required input sequence.

Submission

While submitting to the classroom prepare the report from the third page of this template including the current template in the first and second page. From the third page write your report including but not limited to the following points.

- Attach your entity code and testbench code
- Attach your output visuals
- Challenges faced during the experiment
- Conclusion

accumulator_8bit.v:

```
`timescale 1ns / 1ps
```

```
module accumulator_8bit(
```

```
    input [7:0] A,  
    input [7:0] B,  
    input CLK,  
    input RESET,  
    output reg [7:0] Q
```

```
);
```

```
always @(posedge CLK) begin
```

```
    if (RESET)
```

```
        Q <= 8'b00000000;
```

```
    else
```

```
        Q <= A + B;
```

```
end
```

```
endmodule
```

Circuit Diagram / Schematic:

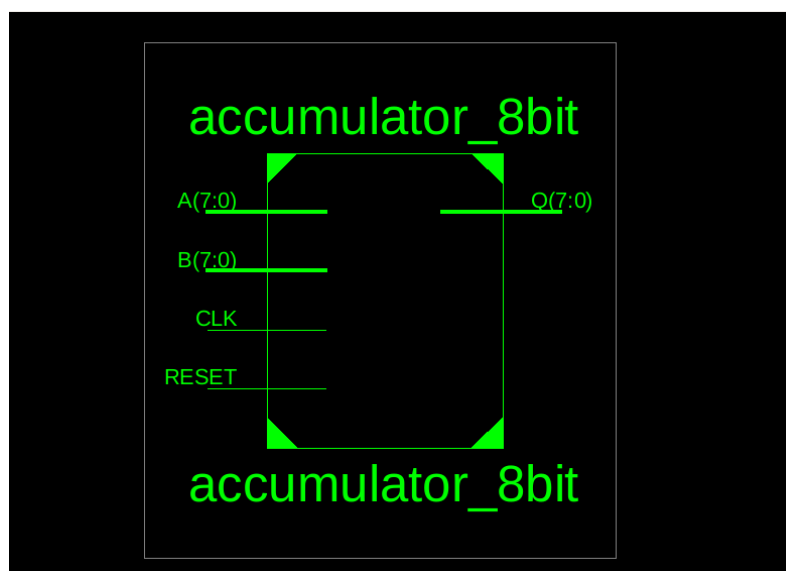


Fig. 1: RTL Schematic of the 8-bit Accumulator

accumulator_8bit_tb.v:

```
`timescale 1ns / 1ps

module accumulator_8bit_tb;

    reg [7:0] A, B;
    reg CLK, RESET;

    wire [7:0] Q;

    accumulator_8bit uut (
        .A(A),
        .B(B),
        .CLK(CLK),
        .RESET(RESET),
        .Q(Q)
    );

    always #5 CLK = ~CLK;

    initial begin

        CLK = 0;
        RESET = 0;
        A = 0;
        B = 0;

        // Reset
        RESET = 1;
        #10;

        // 3 + 5 = 8
        RESET = 0;
        A = 8'b00000011;
        B = 8'b00000101;
        #10;

        // 2 + 1 = 3
        A = 8'b00000010;
        B = 8'b00000001;
        #10;
```

```
// 255 + 1 = 0 (overflow)
A = 8'b11111111;
B = 8'b00000001;
#10;
```

```
// 170 + 5 = 175
A = 8'b10101010;
B = 8'b00000101;
#10;
```

```
$finish;
```

```
end
```

```
endmodule
```

Simulation Waveform:

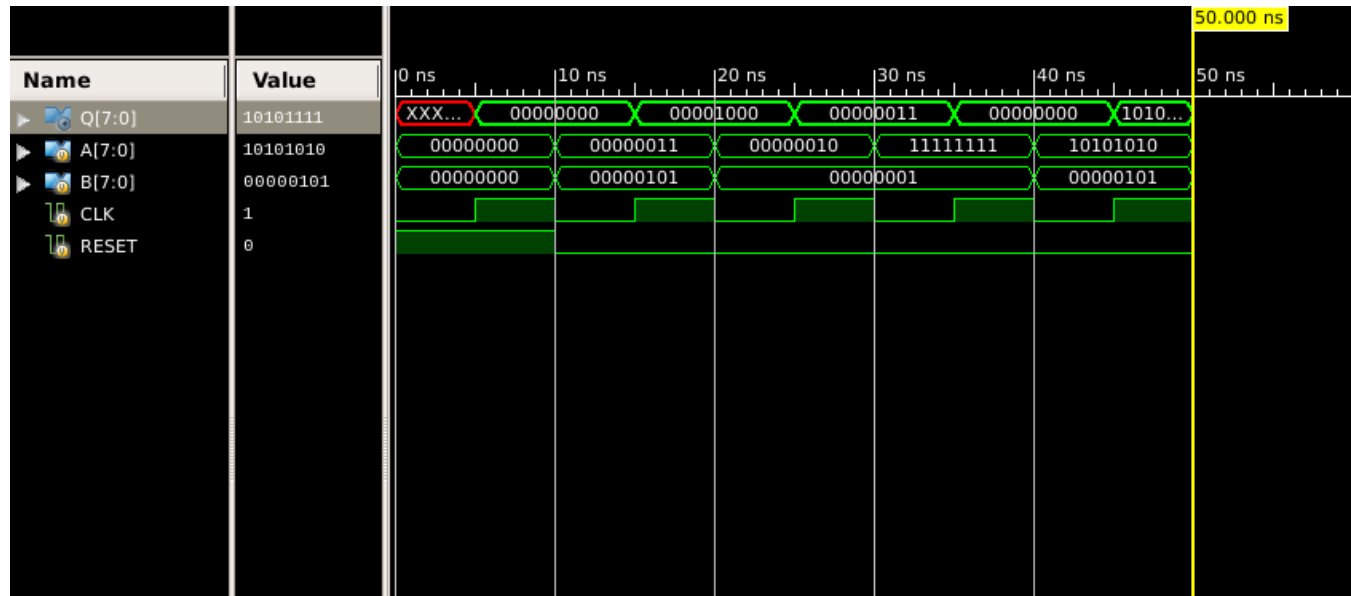


Fig. 2: Simulation Waveform of the 8-bit Accumulator

Table: Simulation Results of the 8-bit Accumulator:

Step	Operation	A	B	Expected Q
1	Reset	00000000	00000000	00000000
2	3 + 5	00000011	00000101	00001000
3	2 + 1	00000010	00000001	00000011
4	255 + 1	11111111	00000001	00000000
5	170 + 5	10101010	00000101	10101111

The simulated output Q matched the expected result for each applied input combination.

Github Repositories:

Commit: Add 8-bit Accumulator design, testbench, RTL schematics, and simulation results

Commit Link: <https://github.com/naima-tabassum/CSE450-VLSI-Lab/commit/e3e3405db4cdf667068fb588f335811838d8c60c>

Challenges faced during the experiment

During the experiment, one of the main challenges was understanding how the 8-bit addition and register operation worked together. It was also important to make sure that the output was stored correctly at each rising edge of the clock. Handling the active-high reset and understanding the overflow case, especially when adding 255 and 1, required some attention. During simulation, the input values and clock timing also had to be set carefully so that the expected outputs could be observed correctly.

Conclusion

The 8-bit accumulator was successfully designed and simulated using Verilog. The simulation showed that the accumulator correctly adds two 8-bit inputs and stores the result at each positive edge of the clock. The synchronous reset also worked correctly by clearing the output to 00000000. In the overflow case of 255 + 1, the output became 00000000 because the extra carry bit was not stored. Overall, the testbench results confirmed that the 8-bit accumulator performed the required operations correctly.

